ניתוח ועיצוב מערכות מידע

דרישות

הרצאה 3

עודכן: 07.11.2026

סדר היום

1. **מהן דרישות**
2. למה זה קריטי
3. זיהוי צרכים
4. ניסוח דרישות
5. תבניות ושלמות
6. סיכום

היכן אנחנו ב-SDLC

שלב ניתוח דרישות — כל שלב במחזור החיים ובעלי התפקידים המעורבים בו

שלבי ה-SDLC — מי מוביל ומי משתתף בכל שלב

מיפוי בעלי התפקידים והאקטורים לאורך מחזור החיים של מערכת המידע

← הזמן מתקדם מימין לשמאל

לחצו על שלב או על תפקיד לקבלת הסבר קצר.

1 ייזום	2 חקר מצב קיים	3 ניתוח דרישות (אנחנו כאן)	4 חקר ישימות	5 עיצוב	6 פיתוח	7 בדיקות	8 הטמעה ותפעול
★ בעל אחריות: PM / Program Manager	★ בעל אחריות: Business Analyst	★ בעל אחריות: Business Analyst	★ בעל אחריות: PM / Program Manager	★ בעל אחריות: Designer / System Analyst	★ בעל אחריות: Developer (FE/BE/DBA/MW)	★ בעל אחריות: QA	★ בעל אחריות: DevOps
הנהלה בכירה והנהלות הארגונים	הנהלה בכירה והנהלות הארגונים	משתמשי מערכת ישירים	הנהלה בכירה והנהלות הארגונים	UX / Creative	Full-stack Developer	Developer (FE/BE/DBA/MW)	ארגוני תמיכה (1st / 2nd / 3rd)
Business Analyst	משתמשי מערכת ישירים	משתמשי מערכת עקיפים	Business Analyst	Data Scientist	Reporting / BI	Full-stack Developer	QA
	משתמשי מערכת עקיפים	הנהלה בכירה והנהלות הארגונים	Designer / System Analyst	Developer (FE/BE/DBA/MW)	DevOps	משתמשי מערכת ישירים	משתמשי מערכת ישירים
	Designer / System Analyst	UX / Creative	Data Scientist	Full-stack Developer	Designer / System Analyst	Business Analyst	משתמשי מערכת עקיפים
	Data Scientist	Designer / System Analyst	DevOps	Reporting / BI			הנהלה בכירה והנהלות הארגונים
	Reporting / BI						

ניתוח דרישות

איפה היינו, ולאן ממשיכים

מ"איך זה עובד היום" אל "מה המערכת החדשה צריכה לעשות"

בשיעור הקודם — הבנו ומידלנו את התהליך הקיים

- לקחנו את תהליך הזמנת האוכל בקמפוס כפי שהוא מתנהל היום
- תיעדנו אותו באופן פורמלי כתרשים BPMN — מי עושה מה, באיזה סדר, ובאילו החלטות
- זה התמונה של ה-as-is: המצב הקיים, לפני שנגענו בו

הגשר: מהבנה אל דרישה

תיאור התהליך הקיים עונה על "איך זה עובד היום". אבל מערכת מידע נבנית כדי לשנות את התהליך — לכן עכשיו השאלה משתנה ל"מה המערכת החדשה צריכה לעשות". זו בדיוק נקודת המעבר אל ניתוח הדרישות.

היום אנחנו נכנסים לשלב ניתוח דרישות — מתרגמים את ההבנה לדרישות למערכת.

ניתוח דרישות

מהן דרישות?

מה המערכת חייבת לעשות — ומה היא חייבת להיות

הגדרה

דרישה היא אמירה על מה שהמערכת חייבת לעשות או להיות — תכונה, יכולת או אילוץ שהמערכת החדשה נדרשת לקיים. הדרישות הן הגשר מהתהליך הקיים שמיפינו (הרצאה 2) אל מה שהמערכת החדשה צריכה לספק.

דרישות פונקציונליות (FR)

מה המערכת עושה

- לעיין בתפריט ולשלוח הזמנה
- לשלם ולקבל התראה כשההזמנה מוכנה
- מנהל מעדכן זמינות פריטים ומחירים
- המטבח מסמן הזמנה כ'מוכנה'

דרישות לא-פונקציונליות (NFR) · Non-Functional Requirement (דרישה לא-פונקציונלית)

כמה טוב היא עושה זאת

- ביצועים — עמידה בעומס שיא (12:00–14:00)
- אבטחה ופרטיות של נתוני הסטודנטים
- שמישות — מסך הזמנה פשוט בתוך הפסקה של 15 דקות
- מדרגיות — תמיכה ב-5000 סטודנטים

ובעולם תפגשו עוד תוויות

"דרישות עסקיות", "דרישות שוק", "דרישות טכניות", "דרישות UX" — חלקן בכלל לא דרישות של המערכת אלא כללי-יסוד של הפרויקט. בשקף הבא: לאן כל תווית הולכת אצלנו.

פונקציונלי = מה המערכת עושה · לא-פונקציונלי = כמה טוב — והדרישות הפונקציונליות הן אלו שנתפוס כתרחישי use case בהרצאה הבאה.

ניתוח דרישות

שני דליים: פונקציונליות — ולא-פונקציונליות

כל תווית שתפגשו במסמכים אמיתיים נופלת לאחד משני הדליים — ורובן: סוג של לא-פונקציונלית · 🖱️ לחצו ② ③ לעוד דוגמאות

הדלי	הסוג	מה זה	דוגמה — ועוד שתיים בלחיצה
פונקציונליות **FR**	**פונקציונלית**	מה המערכת עושה — פעולה של שחקן	"המערכת תאפשר לסטודנט להזמין ולשלם" ② ③
לא-פונקציונליות **NFR** קטגוריית-על — ואלה הסוגים שבתוכה	**ביצועים ועומס**	מהירות ותפוקה — גם בשעת שיא	"תהליך הזמנה מלא ≥ 90 שניות" ② ③
	זמינות ואמינות	כמה מהזמן המערכת למעלה, ומה קורה בנפילה	"זמינות 99.5% בשעות הפעילות" ② ③
	אבטחה ופרטיות	מי נכנס, מה נשמר, מי רואה מה	"אימות דרך חשבון האוניברסיטה בלבד" ② ③
	מדרגיות	גדילה — בלי לבנות מחדש	"תמיכה ב-5000 סטודנטים" ② ③
	שמישות וממשק	קלות שימוש, שפות, נגישות	"הזמנה ראשונה, ללא הדרכה, תוך ≥ 3 דקות" ② ③
	טכניות ותשתית	ה"איך" שנכפה מבחוץ — אילוץ עיצוב, לעולם לא FR	"המערכת תרוץ על שרתי האוניברסיטה" ② ③
	עסקיות — אילוצי פרויקט	כללי-יסוד של הפרויקט, לא של המערכת — מתועדים בחלק א'	"תקציב עד 200 אלף ₪" ② ③

תווית אחת נשארת מחוץ לשני הדליים: דרישות שוק ("המתחרה כבר תומך ב...") — ניהול מוצר (MRD/PRD), לא מסמך הדרישות שלנו; אם רלוונטי — שורה ברקע הפרויקט.

⚠️ אילוץ שהתעלמו ממנו מפיל פרויקט
"עד פתיחת הסמסטר" ו"על התשתית הקיימת" לא יקבלו FR-n — אבל מי שלא תיעד אותם יגלה אותם בכאב. מכירים, מתעדים, ומפנים.

🤖 וכשה-AI מציע "Technical Requirements"
בקשו דרישות ותקבלו בשמחה "React, PostgreSQL". עכשיו יש לכם את המיון: זה הדלי הלא-פונקציונלי — אילוץ, לא FR. ממיינים, לא מוחקים.

FR = מה המערכת עושה · NFR = כמה טוב ובאילו אילוצים — שני דליים, וכל השאר סוגים בתוך השני.

ניתוח דרישות

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Artificial Intelligence = AI · בינה מלאכותית

למה קריטי לנתח דרישות נכון

לתפוס טעות מוקדם — לפני שהיא מתומחרת בייצור

לעשות נכון בפעם הראשונה

דרישה שמנותחת נכון בתחילת התהליך היא הצעד עם המינוף הגבוה ביותר: היא חוסכת את כל העלות של תיקון מאוחר — תכנון מחדש, פיתוח מחדש ובדיקות חוזרות.

עקומת עלות התיקון — פי 10 בכל שלב

דרישות (req)
1×
מתקנים בשורת טקסט במסמך הדרישות

עיצוב (design)
10×
משנים את התרשימים והמודל

פיתוח (dev)
100×
כותבים מחדש קוד שכבר נכתב

ייצור (production)
1000×
תקלה אצל המשתמש — וגם נזק לאמון

דרישה חסרה או שגויה עולה פי 10 לתיקון בכל שלב מאוחר יותר בתהליך.

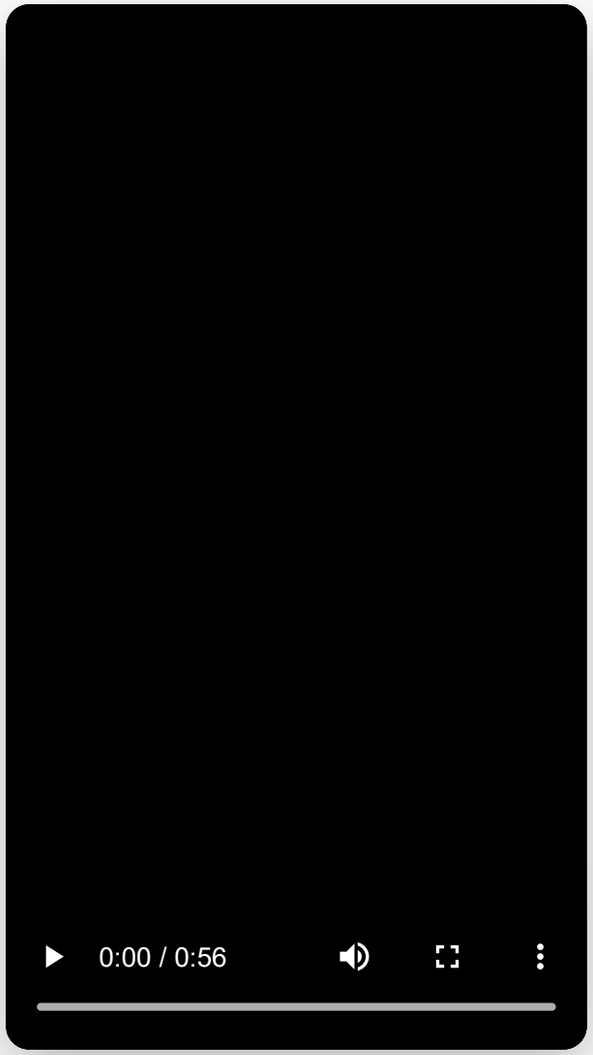
⚠️ מה קורה כשהדרישות לא מוגדרות

בלי דרישות ברורות התהליך גולש: נפיחות היקף (scope creep) — הפרויקט תופח בלי גבול; בונים את המערכת הלא נכונה שלא פותרת את הצורך האמיתי; ומשלמים בעבודה חוזרת (rework) שמייקרת ופוגעת באיכות.

ניתוח דרישות חד מראש = פחות עלות, פחות סיכון ואיכות גבוהה יותר.

ניתוח דרישות

רגע של השראה — לפני שיוצאים להקשיב



ניתוח דרישות

שיטות לזיהוי צרכים

איך מוציאים דרישות מבעלי העניין — לפני שמנסחים אותן

לפני שמנסחים דרישה, צריך להוציא אותה מהאנשים שמכירים את התהליך. לכל שיטה יתרון וחיסרון — בפועל משלבים כמה מהן.

ראיונות

שיחה אחד-על-אחד עם בעל עניין. מעמיק, אבל תלוי במי שואלים, את מי, ועל מה.

שאלונים

מגיעים להרבה אנשים במהירות — אבל אחוז המענה נמוך וקשה להעמיק.

תצפיות

צופים בעבודה בפועל. חושף מה שאנשים עושים — לא רק מה שהם אומרים שהם עושים.

בחירה מקטלוג

בוחרים מתוך קטלוג דרישות שכבר הועלו ע"י העובדים (best practices).

ניתוח מסמכים

קוראים נהלים, טפסים ודוחות קיימים — התהליך הנוכחי כבר מתועד בהם.

סדנאות ואבטיפוס

מפגישים בעלי עניין סביב מודל חלקי (prototype), כדי לדייק צרכים מוקדם.

⚠ זיהוי צרכים זה קשה

בעלי העניין לא תמיד יודעים לנסח מה הם צריכים: דרישות מנוגדות בין אנשים שונים, צרכים מעורבבים עם פתרונות, ופוליטיקה ארגונית. הרבה מערכות נכשלות לא בגלל קוד גרוע, אלא בגלל שהן לא עונות על הצרכים האמיתיים של המשתמשים.

בדיוק כאן נכנסת מסגרת מובנית — Wetherbe (בשקפים הבאים) · ואל השיטות עצמן נחזור מיד אחריה: איך מבצעים כל אחת.

ניתוח דרישות

המאמר: Wetherbe — לקבוע דרישות נכון

"Executive Information Requirements: Getting It Right", MIS Quarterly, 1991 — חומר הקריאה של השיעור

סיפור הפתיחה — 32 הדוחות

סמנכ"ל הנדסה חדש מקבל אוטומטית 32 דוחות בחודש ולא מבין מה עושים עם רובם. הוא מבקש את רשימות התפוצה כדי לשאול בשקט את המקבלים האחרים למה הם משמשים. התשובה: יש בדיוק שני מקבלים — הוא והעוזר שלו (עותקי גיבוי). אחרי שנה של מעקב: הוא צריך 4 דוחות מתוך ה-32 — ומידע חשוב אחד שאף דוח לא סיפק.

מה הסיפור מלמד — נקודת המוצא של המאמר

מנהלים לא יודעים איזה מידע הם צריכים. זו לא עלבון — זו תכונה של זיכרון וקוגניציה אנושית (עוד מ-Ackoff, 1967). ולכן לשאול אותם ישירות — לא יעבוד. המשימה החשובה ביותר של מנהל היא קבלת החלטות, והמשאב המרכזי שלה הוא מידע.

וזה לא מקרה אחד — הממצא המרכזי

76 / 76 מחקר של מרכז ה-MIS באוניברסיטת מינסוטה: 76 מערכות מידע ב-26 ארגונים — כולן נדרשו לתיקונים אחרי שכבר "הושלמו", רק כדי להתקרב לצורכי ההנהלה. ותיקון אחרי השלמה עולה פי 50–100 מתיקון בשלב העיצוב (חדר אמבטיה בבית בנוי לעומת בשרטוט).

⚠️ עדיין לא מאמינים שמאמר מ-1991 רלוונטי?

סקר עולמי 2016: "דרישות חסרות / נסתרות" — עדיין בעיה מס' 1 (48% מהחברות) · 2020: רק 31% מהפרויקטים מצליחים · PMI: 37% מהארגונים מסמנים איסוף דרישות שגוי כגורם הכישלון המרכזי.

זו לא בעיית טכנולוגיה — זו בעיית קביעת דרישות. המאמר: 4 טעויות ← 4 פתרונות, בשקפים הבאים.

ניתוח דרישות

מה מנתחים מנסים — ולמה זה נכשל

שלוש ה"תוכניות" הקלאסיות של מנתח המערכות, לפי המאמר — וכשל אחד משותף

תוכנית א' — לשאול ישירות

"איזה מידע אתם צריכים מהמערכת החדשה?" — המנהלים עונים כמיטב יכולתם, בהנחה ש"קוסמי המחשבים" כבר יסתדרו.

⛔ **חודשים ומיליונים אחר-כך מתברר: זה לא מה שהם צריכים.**

תוכנית ב' — חתימת משתמש

מחתימים את המנהלים על מסמך הדרישות, כדי "לחייב" אותם לקבל את מה שביקשו.

⛔ **"פיסת נייר חסרת אונים מול זעם ההנהלה."**

בנק גדול ביטל פרויקט של $40M — ההנהלה, שחתמה: "אתם הטכנולוגים הייתם צריכים להגן עלינו מחוסר המומחיות שלנו."

תוכנית ג' — קטלוג דוחות

מציגים למנהל קטלוג דוחות מוכנים — שיבחר מה שהוא צריך.

⛔ **מנהלים לוקחים הכול, "ליתר ביטחון".**

הניסוי: הוצעו 20 דוחות, רק 8 מועילים (לפי פאנל מומחים) ו-12 חסרי-ערך בכוונה — רוב המנהלים לקחו את כל ה-20. כך נולד עומס המידע.

⚠️ הכשל המשותף

שלוש התוכניות מניחות שמישהו — המנהל או הקטלוג — כבר יודע את התשובה. אבל נקודת המוצא של המאמר היא שאף אחד לא יודע אותה מראש; צריך לחלץ אותה. איך? ארבע הטעויות וארבעת הפתרונות — בשקפים הבאים.

אם השיטה שלכם היא "לשאול מה רוצים ולהחתים" — אתם בתוכנית א' + ב'. המאמר מציע דרך אחרת.

ניתוח דרישות

טעות 1 — מערכת "של מחלקה" במקום חוצת-פונקציות

רוב המידע שמשפר החלטה בתוך מחלקה — מגיע מחוץ לה

🧩 על מה הטעות הזו? תכולת המערכת — איזה מידע ואילו פונקציות נמצאים בפנים. (איך יושבים עם האנשים — זו הטעות הבאה.)

הסיפור: הקצאת מלאי במחסן (FIFO)

למחסן 5 הזמנות ומלאי רק ל-3. לפי המידע שיש למחסן, ההחלטה ההוגנת היא "כל הקודם זוכה" (FIFO). התוצאה: הלקוח הגדול — שההזמנה דחופה לו, שכבר קיבל הזמנה באיחור וזעם, שמשלם בזמן, ושמשאית ממילא נוסעת אליו היום — לא קיבל. קיבל מי שכמעט לא קונה, לא ממהר, ולא משלם בזמן.

המידע שהיה הופך את ההחלטה — וכולו מחוץ למחסן

חשיבות הלקוח — ממכירות | דחיפות ההזמנה — ממכירות | רווחיות ההזמנה — ממכירות/כספים | מצב אשראי — ממחלקת אשראי | לוח משאיות — ממשלוחים

הפתרון: עיצוב חוצה-פונקציות

כל פונקציה שהתהליך נוגע בה משתתפת בעיצוב המערכת, והמידע זורם ביניהן. "מידע הוא כוח" — ולכן משתפים אותו, כדי להעצים את מקבלי ההחלטות; ארגון שלא משתף נשאר עם יד שמאל שלא יודעת מה עושה יד ימין.

אצלנו בקמפוס

החלטת הדלפק "למי למסור" נשענת על מידע מהמטבח (מה מוכן) ומההזמנות (מי הזמין); ההתראה לסטודנט (FR-2) נולדת מסימון של המטבח. המערכת שלנו חוצת-פונקציות מיומה הראשון.

עדיין נכון — 2016: תקשורת לקויה בין הצוות ללקוח היא בעיה מס' 2 בעולם (41% מהחברות).

לפני שכותבים דרישות: מי עוד נוגע בתהליך — ואיזה מידע שלו ישפר החלטות של אחרים?

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית

טעות 2 — ראיונות אישיים במקום סדנה משותפת

זיכרון עובד בקבוצה — וכל מחלקה מגיעה עם אג'נדה משלה

על מה הטעות הזו? שיטת האיסוף — מי יושב בחדר כשמגדירים את הדרישות. טעות 1 עסקה בתכולה (מה בתוך המערכת); אפשר לתקן אותה — לעצב חוצה-פונקציות — ועדיין לראיין אחד-אחד, ולהפסיד את החדר.

ניסוי 10 הבדיחות

בקשו ממנהל, לבדו, לספר 10 בדיחות — הוא ייתקע אחרי שתיים-שלוש. שימו את אותם מנהלים יחד בחדר — הקבוצה תגיע ל-80–100 בדיחות, וכל אחד יזהה 80–90% מהן.

אותו דבר עם דרישות: לבד, מנהל נזכר רק במה שהיה צריך לאחרונה. הזיכרון הקבוצתי שולף גם את השאר.

ולכל מחלקה — אג'נדה

באותו תהליך הזמנות: מכירות רוצות אספקה מהירה ללקוח · אשראי רוצה גבייה מלאה · המחסן רוצה עלויות מלאי נמוכות · המשלוחים רוצים ניתוב חסכוני.

בראיונות נפרדים כל אחד מושך לכיוונו — מטרה משותפת נולדת רק כשכולם שומעים זה את זה.

הפתרון: סדנת JAD (Joint Application Development · פיתוח יישומים משותף) — Joint Application Design

כל בעלי-העניין באותו חדר, באותו זמן, מגדירים את הדרישות יחד: זיכרון משותף, אג'נדות על השולחן, ודרישה אחת מוסכמת במקום סבב ראיונות וסתירות.

התובנה שנולדה רק בחדר משותף — חברת קטלוגים

מחלקת האשראי דרשה עוד ועוד בדיקות אשראי (יעד: אפס הפסדי גבייה). בסדנה המשותפת התברר שעדיף לא לבדוק אשראי בכלל: שולחים ללקוח חדש קטלוג של פריטים זולים בלבד; מי ששילם — משודרג. ההתנהגות בפועל מנבאת טוב יותר מכל בדיקה, ועולה פחות. לבד, אשראי לעולם לא היה מגיע לזה — כי לפי האג'נדה שלו, המסקנה "אל תבדקו" היא כישלון.

עדיין נכון — מאז 1994 ועד היום: "מעורבות משתמשים" היא גורם ההצלחה מס' 1 בדוחות CHAOS.

דרישה טובה היא הסכמה בין מחלקות — לא ממוצע של ראיונות נפרדים.

ניתוח דרישות

טעות 3 — שואלים את השאלה הלא-נכונה

"איזה מידע אתם צריכים?" ≈ פסיכולוג ששואל מטופל "איזה טיפול אתה צריך?"

⛔ מוכר "לוקח הזמנות" — מכסחת Toro

"איזה רוחב להב? כמה כוחות סוס? צמיגים רחבים או צרים? שק אחורי או צדי?" — הקונה לא יודע לענות, המוכר מגלגל עיניים... ודוחף את היחידה של $2,800.

✅ מוכר "פותר בעיות" — שאלות עקיפות

"כמה גדולה החצר? משופעת? מה פני השטח? יש עצים וגדר? גם אשתך תשתמש?" — ומהתשובות נגזרת התצורה הנכונה: $1,800, מנומקת סעיף-סעיף.

הפתרון: ראיון מובנה — שאלות עקיפות שמגיעות אל הדרישות "מאחור"

לא שואלים "איזה מידע?" — שואלים על העבודה, ושלושה סטים של שאלות ממפים אותה למידע:

BSP · בעיות והחלטות (IBM)

מהן הבעיות בתהליך? → מה פותר אותן? → איזה מידע משרת את הפתרון?
ובמקביל: מהן ההחלטות שלך? → איזה מידע ישפר כל החלטה?

CSF (Critical Success Factors · גורמי הצלחה קריטיים) **· גורמי הצלחה (Rockart)**

מהם 4–8 גורמי ההצלחה הקריטיים של היחידה שלך? → איזה מידע יראה שכל אחד מהם בשליטה? (דוחות סיכום וחריגים.)

E/M · תוצרים ואמצעים (Wetherbe & Davis)

מה התוצר, ומה עושה אותו אפקטיבי ללקוח? → איזה מידע ימדוד זאת?
מה התהליך, ומה זו יעילות בו? → איזה מידע ימדוד אותה?

📖 רשת ביטחון מובנית: מה שלא נזכר כ"בעיה" — צץ שוב כ"החלטה" או כ"גורם הצלחה". החפיפה היא הפיצ'ר.

עדיין נכון — מטא-אנליזה (IEEE TSE, 2010): ראיון מובנה הוא טכניקת האיסוף האפקטיבית ביותר; ב-2020 עוד מפתחים פרוטוקולי אימון לטעויות מראיינים.

את הראיונות בקמפוס ננהל בדיוק כך — ותתאמנו על זה מול מרואיין AI בכלי הקורס.

ניתוח דרישות

Artificial Intelligence = AI · בינה מלאכותית

טעות 4 — בלי ניסוי וטעייה

דרישות מפורטות לא נקבעות על הנייר — לומדים אותן מול מסך חי

ככה בני אדם פותרים בעיות

מודדים בגדים לפני שקונים · נסיעת מבחן לפני רכב · מזיזים רהיטים כמה פעמים · ויותר מחור-מסמר אחד בקיר עד שהתמונה ישרה. רק בדרישות מפורטות — אילו מסכים, אילו דוחות, אילו שדות — מצפים מהמנהל לדייק על הנייר, בניסיון הראשון, בלי לראות כלום.

הפתרון: אב-טיפוס — תוך ימים, לא חודשים

יומיים עד חמישה ימים אחרי הראיון המובנה, המנהל כבר יושב מול אב-טיפוס חי ומגיב למשהו מוחשי במקום לדמיין מופשט. הראיון נותן את המושגים; האב-טיפוס מדייק את הפרטים.

אב-טיפוס ראשוני ← המנהל מתנסה ← משפרים: נתונים · מסכים · דוחות ← מקובל? רק אז מפתחים באמת

⚠️ ומה עם תקציב ולוח-זמנים?

נקבעים אחרי שיש אב-טיפוס מוסכם — לא לפניו. קבעתם מחיר ותאריך-כניסה לבית לפני שראיתם בתים? כשמקבעים תקציב לפני שיודעים מה בונים, הדבר היחיד שנשאר "לגמש" הוא התוכן — וכך פונקציונליות קריטית נשארת בחוץ.

הפתרון שניצח — הפך ל-agile ו-MVP. ובכל זאת רק 31% הצלחה (2020), כי שלוש הטעויות האחרות הן בעיות של אנשים.

בהרצאה 4-5 נראה עוד כוח של האב-טיפוס: הוא שואל בשבילכם את שאלות ה"מה אם" שאף אחד לא חשב עליהן.

ניתוח דרישות

Wetherbe בשקף אחד — 4 טעויות, 4 פתרונות

מה שהמאמר מבקש שתיקחו איתכם — והסיפור שמזכיר כל שורה

#	הטעות	הפתרון	הסיפור
1	מערכת "של מחלקה" — מבט פונקציונלי צר	**עיצוב חוצה-פונקציות — כל מי שהתהליך נוגע בו משתתף**	הקצאת FIFO במחסן
2	ראיונות אישיים, אחד-אחד	**סדנת JAD — כולם באותו חדר, באותו זמן**	10 בדיחות · חברת הקטלוגים
3	השאלה הלא-נכונה: "איזה מידע אתם צריכים?"	**ראיון מובנה בשאלות עקיפות — BSP · CSF · E/M**	מכסחת הדשא של Toro
4	בלי ניסוי וטעייה בעיצוב המפורט	**אב-טיפוס תוך ימים; תקציב ולו"ז — אחריו**	חיפוש בית

הבינה בראיון

Claude מנסח מראש שאלות עקיפות בשלושת הסטים לכל בעל-עניין, ומסכם סדנת JAD לרשימת דרישות — בתוך דקות. גם המחקר ב-2024 בונה מרואייני-AI לחילוץ דרישות — בדיוק הכלי שתתאמנו עליו.

⚠ אילו שאלות היא לא שאלה?

אם הבינה כתבה את השאלות — מי מוודא שהיא לא נפלה בדיוק בטעות 3? בדקו איזו זווית (בעיה, החלטה, גורם הצלחה) נשארה בחוץ.

1991: כל 76 המערכות נדרשו לתיקונים · 2020: רק 31% מהפרויקטים מצליחים — שלושים שנה, אותם ארבעה שיעורים.

ניתוח דרישות

CSF = Critical Success Factors · גורמי הצלחה קריטיים · JAD = Joint Application Development · פיתוח יישומים משותף · AI = Artificial Intelligence · בינה מלאכותית

חוזרים לארגז הכלים — ואיך עושים את זה בפועל?

בשקף 7 מנינו שיטות; Wetherbe כבר נתן לנו שלוש — את השאר נלמד עכשיו, שיטה לשקף

✓ כבר בידיים — הפתרונות של Wetherbe

ראיון מובנה — מה לשאול
שלושת סטי השאלות העקיפות (BSP · CSF · E/M) — טעות 3.

סדנת JAD
כולם באותו חדר, זיכרון קבוצתי, אג'נדות על השולחן — טעות 2.

אב-טיפוס
דרישות מפורטות לומדים מול מסך חי, תוך ימים — טעות 4.

← עכשיו נעמיק — איך מבצעים

ראיון — איך מנהלים את הפגישה
לפני · במהלך · אחרי — מי, איך מקשיבים, ואיך מאמתים.

שאלונים
להגיע ל-5000 סטודנטים: מתי זה עובד, ואיך לא ליפול בניסוח.

תצפיות
מה שעושים ≠ מה שאומרים — לצפות בעמדה בשעת העומס.

מסמכים • קטלוג
לקרוא מה שכבר קיים — ולמה קטלוג הוא כלי שלכם, לא תפריט ללקוח.

אף שיטה אינה מספיקה לבדה — משלבים: מסמכים לפני הראיון, תצפית אחרי, שאלון לכימות, סדנה להכרעה.

ניתוח דרישות

CSF = Critical Success Factors · גורמי הצלחה קריטיים · JAD = Joint Application Development · פיתוח יישומים משותף

ראיון — איך מנהלים את הפגישה

מה לשאול כבר יש לכם (BSP · CSF · E/M) — עכשיו: איך מריצים ראיון שמפיק דרישות

לפני

- מי? כיסוי תפקידים — לא רק מנהלים: מי שעושה את העבודה בפועל (דלפק, מטבח).
- מטרה לראיון — איזו החלטה/תהליך רוצים להבין הפעם.
- לקרוא קודם את המסמכים — לא שורפים פגישה על מה שכתוב בנוהל.
- להכין את סטי השאלות העקיפות מראש — בשקף הבא: מוכנים לקמפוס.

במהלך

- משפך: פתוחות ("ספר לי על צהריים עמוסים") → ממוקדות.
- להקשיב 80/20 — המרואיין מדבר, אתם רושמים דוגמאות ומספרים.
- "תראה לי" — בקשה אחת שהופכת ראיון לתצפית מיני.
- מילה מסוכנת של המרואיין ("מהר", "בדרך כלל", "כולם") = שאלת חידוד מיד.

אחרי

- לסכם תוך 24 שעות — אחר-כך הפרטים מתאדים.
- לשלוח לאישור המרואיין — "זה מה שהבנתי; מה פספסתי?" (אימות!).
- סתירות מול ראיונות אחרים → לא מכריעים לבד: סדנה או ראיון המשך.
- כל צורך שזוהה → טיוטת דרישה עם מקור.

⚠️ הטעות הנפוצה

לראיין רק את המנהל. הוא יודע את המדיניות — אבל את המעקפים, החריגים והכאב האמיתי יודע מי שעומד בדלפק ב-12:30. ראיינו את שניהם — והשוו.

ראיון טוב = הכנה + הקשבה + אימות בכתב. הקסם אינו בפגישה — הוא לפניה ואחריה.

ניתוח דרישות

CSF = Critical Success Factors · גורמי הצלחה קריטיים

מכינים את השאלות — שלוש מסגרות חשיבה על אותה קפיטריה

לא בוחרים "שיטה" — אלו שלוש זוויות שמחליפים ביניהן תוך כדי הראיון · לחצו על כרטיס לשאלות המוכנות

BSP • בעיות והחלטות (IBM)

Business Systems Planning · תכנון מערכות עסקיות

מתחילים מהכאב של היום

מהן הבעיות? מה יפתור אותן? אילו החלטות אתה מקבל? — ומאחורי כל תשובה: איזה מידע נדרש?

לחצו לשאלות המוכנות

CSF • גורמי הצלחה (Rockart)

Critical Success Factors · גורמי הצלחה קריטיים

מתחילים ממה שחייב להצליח

4–8 גורמים קריטיים — ואיזה מידע יראה שכל אחד מהם בשליטה? מוליד דרישות ניטור ו-NFR. הזווית שתפגשו הכי הרבה בתעשייה — מדדי ה-KPI שכל ארגון מנהל היום הם הצאצא הישיר שלה.

כך זה נראה בארגון — לוח מחוונים

לחצו לשאלות המוכנות

E/M • תוצרים ואמצעים (Wetherbe & Davis)

Ends / Means Analysis · ניתוח תוצרים ואמצעים

מתחילים מהתוצר ומהתהליך

מה מקבל הסטודנט ומה עושה את זה אפקטיבי? מה התהליך ומה זו יעילות בו? — והמידע שמודד את שניהם.

לחצו לשאלות המוכנות

נתקעתם? מחליפים זווית — לא שיטה. מה שנשכח כ"בעיה" יצוץ כ"גורם הצלחה" או כ"מדד יעילות" — החפיפה המכוונת היא רשת הביטחון.

ניתוח דרישות

CSF = Critical Success Factors · גורמי הצלחה קריטיים · NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · FR = Functional Requirement · דרישה פונקציונלית

📋 שאלונים — להגיע לרבים

שאלון לא מגלה צרכים — הוא מכמת ומתעדף את מה שהראיונות כבר גילו

מתי שאלון הוא הכלי הנכון

- הרבה משיבים שאי-אפשר לראיין — 5000 סטודנטים.
- לכמת: "כמה חשוב לך סינון תזונתי?" (סולם 1–5) → נתונים ל-MoSCoW.
- להכריע בין חלופות שהראיונות העלו — לא להמציא חדשות.
- כשצריך אנונימיות — על כאבים שלא אומרים בפנים.

איך בונים אותו נכון

- קצר — 5 דקות מקסימום; כל שאלה מיותרת מורידה מענה.
- סגורות + סולם; מעט פתוחות — ובסוף.
- בלי שאלות מטות: "האם לא היית רוצה ש...?" מזמין "כן".
- פיילוט על 5 אנשים לפני הפצה — שאלה דו-משמעית מתגלה מיד.

⚠️ שני הפחים של השאלון

אחוז מענה נמוך — ומי שכן עונה אינו מדגם מייצג (המרוצים מאוד והכועסים מאוד). ואי-אפשר לשאול "למה" — אין שאלת המשך. לכן: שאלון אחרי ראיונות, לעולם לא במקומם.

בקמפוס: ראיונות גילו את המועמדים ← שאלון ל-5000 מכמת ← העדיפויות בטבלת המעקב מגובות בנתונים.

ניתוח דרישות

תצפיות — מה שעושים, לא מה שאומרים

בראיון מתארים את התהליך הרשמי; בפועל עובדים אחרת — הפער הזה הוא מכרה הדרישות

איך צופים נכון

- בשעת העומס — בקמפוס: לשבת ליד הדלפק 12:00–14:00, לא ב-10 בבוקר.
- לתעד צעדים, זמנים והפרעות: כמה זמן לוקח לוודא הזמנה? איפה התור נתקע?
- פסיבית קודם; שאלות — אחרי, לא באמצע העומס.
- כמה ביקורים, בימים שונים — יום אחד הוא אנקדוטה.

מה מחפשים: מעקפים

- פתק דבוק על המסך · אקסל צדדי · "רגע, אני שואל את דנה".
- כל מעקף = דרישה שהמערכת הנוכחית לא עונה עליה — ואיש לא יספר עליו בראיון (הוא "לא לפי הנוהל").
- צעד שכולם מדלגים עליו = דרישה מתה; צעד ידני חוזר = מועמד לאוטומציה.

⚠️ אפקט הות'ורן (Hawthorne)

כשצופים בי — אני עובד "לפי הספר". לכן: להישאר מספיק זמן שישכחו מכם, לחזור כמה פעמים, ולהצליב עם מה שהמסמכים והראיונות אומרים. שלושת המקורות לא מסכימים? שם חופרים.

הדרישה הכי חשובה בקפיטריה לא תיאמר בראיון — היא דבוקה בפתק על הקופה.

ניתוח דרישות

ניתוח מסמכים — התהליך כבר כתוב (חלקית)

לפני שמדברים עם אנשים — קוראים מה שהארגון כבר תיעד; זול, זמין, ולא משנה התנהגות

מה אוספים

- טפסים — נייר ודיגיטליים.
- דוחות קיימים (מי מקבל? זוכרים את 32 הדוחות...).
- נהלים והנחיות עבודה.
- מסכי המערכת הישנה.
- יומן תקלות ותלונות — ארכיון הכאב.

מה מחפשים — וכל פריט מזין תוצר שכבר הכרנו

- שדות בטופס → מועמדים למילון הנתונים (חובה? ייחודי? פורמט?).
- חותמות ואישורים → חוקים עסקיים (מי מוסמך למה — BR!).
- שדה שתמיד ריק → דרישה מתה; לא לגרור אותה למערכת החדשה.
- הערות בעט על הטופס → המעקפים — מה שהטופס לא נתן ואנשים הוסיפו בכוח.
- תלונות חוזרות → דרישות ותרחישי חריג מוכנים.

⚠️ המסמך משקר יפה

מסמכים מתארים את התהליך הפורמלי ומתעדכנים לאט — הנוהל אומר "אישור מנהל לכל החזר", ובפועל דנה מאשרת בווטסאפ. מסמכים = נקודת פתיחה מצוינת לראיון ("ראיתי בנוהל ש... זה באמת כך?") — לא אמת סופית.

קראו לפני שאתם שואלים: תגיעו לראיון עם שאלות חכמות — ועם חצי מילון נתונים ביד.

ניתוח דרישות

קטלוג ושימוש חוזר — כלי חד, בזהירות

דרישות ממערכות דומות, תקני תעשייה, רשימות NFR — למה להמציא מאפס?

✅ השימוש הנכון — צ'ק-ליסט של האנליסט

- בדיקת כיסוי: "לכל מערכת הזמנות יש ביטול, החזר, התראות — איפה זה אצלנו?"
- בדיוק כמו מטריצת ה-CRUD (Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה) ומשפחות ה-NFR — רשימה שמזכירה מה שכחתם.
- דרישות רגולציה וסטנדרטים — לא ממציאים, מאמצים.
- כל פריט שנלקח עובר התאמה: שאלות החידוד + "האם רלוונטי אצלנו?"

⛔ המלכודת — תוכנית ג' של Wetherbe

- להגיש למנהל קטלוג ולומר "סמן מה שאתה צריך" — הוא ייקח הכול.
- ניסוי 20 הדוחות: 12 חסרי-ערך בכוונה — רובם נלקחו "ליתר ביטחון".
- כך נולד עומס המידע של סיפור 32 הדוחות.
- ההבדל כולו: הקטלוג שואל אתכם ("מה חסר לנו?") — לא את המנהל ("מה בא לך?").

🤖 ה-AI הוא קטלוג-העל

בקשו מ-Claude "דרישות למערכת הזמנות" — תקבלו שפע דרישות "סטנדרטיות" ומשכנעות. אותו כלל בדיוק: צ'ק-ליסט לאנליסט, לא תפריט ללקוח — כל הצעה עוברת את שאלות החידוד, את גבולות המערכת, ואת "מי בכלל ביקש את זה?"

קטלוג טוב חוסך המצאת גלגל — קטלוג בידיים הלא-נכונות ממציא דרישות שאיש לא צריך.

ניתוח דרישות

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · AI = Artificial Intelligence · בינה מלאכותית

עקרונות בזיהוי דרישות

מה לזכור כשמזהים צרכים — לפני שעוברים לניסוח

- יש לזהות ולגלות צרכים — לא לאסוף דרישות. מפרידים בין הצורך לבין הפתרון: מתעדים מה חסר, לא איך לפתור
- דרישות טובות דורשות מעורבות הלקוח — מערבים את המשתמשים האמיתיים, אלה שיעבדו עם המערכת
- מבחינים בין דרישה הכרחית לבין דרישה רצויה (nice-to-have) — לא הכל נכנס לסל
- כל דרישה צריכה להתחבר למטרה עסקית — אם אי אפשר לשייך אותה ליעד, סימן שאלה
- דרישה צריכה להיות ניתנת לבדיקה — אם אי אפשר למדוד אם היא התקיימה, היא לא דרישה אלא משאלה
- זיהוי הצרכים הוא תהליך — לא אירוע חד-פעמי: דרישות לעולם אינן מושלמות, ושינויים בהן בלתי נמנעים

⚠ הסכנה החוזרת

דרישות שאינן מוגדרות נכון יובילו לכישלון הפרויקט. הטעות הנפוצה: לרשום פתרון ("SQL", "אפליקציה") במקום את הצורך — וכך לקבע החלטה לפני שהבנו את הבעיה.

אז אספנו דרישות — עכשיו מה?

הדרישות שנאספו הן רק טיוטה ראשונית. עכשיו צריך לעבור עליהן, לחדד, לדייק ולנסח מחדש — כדי להפוך אותן לדרישות ברורות, מדידות, ישימות ומבוססות צורך. בשלב הבא נראה איך עושים זאת בשיטת SMART (Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב).

אספנו צרכים גולמיים — השלב הבא הוא לנסח אותם נכון.

ניתוח דרישות

SMART — ככה כותבים דרישה

חמישה קריטריונים לדרישה ברורה שאפשר לבדוק

המוסכמה: כל דרישה נכתבת בלשון "המערכת ת..." (The system shall...) — המערכת היא הנושא, והמשפט מחייב: מה היא תעשה, לא תיאור כללי.

Specific · S
ספציפית

חד-משמעית, בלי עמימות: "המערכת תאפשר לסטודנט להזמין ארוחה ולשלם בכרטיס סטודנט" — ולא "המערכת תטפל בהזמנות".

Measurable · M
מדידה

אפשר למדוד אם התקיימה: "תתריע תוך 10 שניות מסימון 'מוכן'" — מספר שאפשר לבדוק, לא "מהר".

Achievable · A
בת-השגה

ריאלית במשאבים ובזמן: התראת "מוכן לאיסוף" — אפשרי; "תחזה מה הסטודנט ירצה מחר" — לא לגרסה הראשונה.

Relevant · R
רלוונטית

תורמת למטרת המערכת: תזכורת לאסוף הזמנה — כן; ניהול מלאי הספקים — מחוץ לגבולות המערכת.

Traceable · T
עקיבה

אפשר לעקוב ממנה אחורה — למקור (מי ביקש ולמה), וקדימה — לבדיקה ולתרחיש שיממש אותה. המזהה (FR-n) והשחקן עושים בדיוק את זה.

❌ דרישה פונקציונלית מעורפלת

"המערכת תעדכן את הסטודנט על מצב ההזמנה."

"תעדכן" — מתי? על מה בדיוק? אין טריגר, אין מדד, אין מזהה — אי-אפשר לבדוק וגם לא לעקוב.

✅ אותה דרישה — SMART ועקיבה

"FR-2.1: המערכת תתריע לסטודנט שהזמנתו מוכנה לאיסוף, תוך 10 שניות מרגע שהמטבח סימן אותה כ'מוכנה'."

S — שחקן וטריגר מוגדרים · M — 10 שניות · A — אפשרי בגרסה 1 · R — בלב מטרת המערכת · T — המזהה FR-2.1 עוקב אחורה להורה FR-2 ולמי שביקש, וקדימה — לבדיקה.

דרישה טובה היא דרישה שאפשר לבדוק — אם אי-אפשר למדוד אותה, היא עוד לא SMART.

FR = Functional Requirement · דרישה פונקציונלית · SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

ניתוח דרישות

מילים 'מסוכנות' בדרישה

למה מילה אחת מעורפלת הופכת דרישה שלמה לבלתי-ניתנת לבדיקה

⛔ מילים שמרסקות דרישה

- מהיר (fast) — ביחס למה? אין מספר למדוד.
- ידידותי למשתמש (user-friendly) — לפי מי? אין קריטריון.
- יעיל (efficient) — בזמן? בעלות? לא ברור מה נבדק.
- וכו' (etc.) — מסתיר דרישות שלא נכתבו.
- בערך (approximately) — מספר שאינו מספר.
- ו/או (and/or) — שתי דרישות במשפט, ולא ברור איזו חובה.
- לפי הצורך (as needed) — של מי, ומתי? דוחה את ההחלטה.

למה כל אלו מסוכנות

המכנה המשותף: אי-אפשר לבדוק אם הדרישה התקיימה. כל מילה כזו פותחת פער פרשנות — המנתח חושב דבר אחד, המפתח דבר אחר, והבודק לא יודע מה לבדוק.

פער שמתגלה רק בקבלה עולה הרבה יותר מתיקון מילה אחת עכשיו.

⚠ מבחן פשוט
אם אי-אפשר לכתוב תרחיש בדיקה שעובר או נכשל — הדרישה עוד לא מנוסחת.

איך כן לנסח דרישה

קונקרטי במקום מעורפל · מדיד — מספר, יחידה ויעד · דרישה אחת למשפט (לא "ו/או") · בלשון המחייבת — "המערכת ת..." (The system shall), לא "הזמנה תטופל".

⛔ ניסוח מסוכן

"המערכת תטפל בהזמנות במהירות וביעילות לפי הצורך."

"במהירות", "ביעילות", "לפי הצורך" — אף אחת לא מדידה, וכמה דרישות במשפט אחד.

✅ אותו דבר, מנוסח נכון

"המערכת תקלוט ותאשר הזמנה שנשלחה מהאפליקציה תוך 3 שניות ב-95% מהמקרים בשעת העומס (12:00–14:00)."

קונקרטי, מדיד (95%, 3 שניות), דרישה אחת, "המערכת ת..." — אפשר לכתוב לה תרחיש בדיקה.

המילים שאתם לא כותבים חשובות כמו אלו שכן — כל מילה מעורפלת היא באג שמחכה.

ניתוח דרישות

גם דרישה SMART עוד לא "גמורה"

כל דרישה מסתירה החלטה שטרם התקבלה — המיומנות היא לדעת איזו שאלה חושפת אותה

מי · בעלות מי מבצע כל חלק?	מקור מאיפה הנתון מגיע?	יעד למי הפלט נשלח?	משמעות הפועל מה בדיוק "להסיר"?	תזמון · טריגר מתי, ומה מפעיל?	אמינות מה קורה בכשל?

תרגול חי — דרישות שנשמעות גמורות, והשאלה שמחדדת כל אחת

"המערכת תתריע 'מוכן לאיסוף' תוך 10 שניות"	← **ומה אם ההתראה נכשלה — יש גיבוי? מה נחשב "נמסרה"?**	אמינות
"המערכת תאמת איסוף בקוד חד-פעמי בן 6 ספרות"	← **איפה הקוד מופיע, מי מייצר אותו, ומי מקליד — הסטודנט או הדלפק?**	מקור · בעלות
"המערכת תסיר פריט שאזל מהתפריט"	← **"הסרה" = נעלם לגמרי, או מוצג כ"לא זמין"? שתי מערכות שונות.**	משמעות הפועל
"המערכת תפיק דוח מכירות בסוף היום"	← **נוצר איפה, נשלח למי, ולמה סוף-יום ולא לפי דרישה?**	מקור · יעד · תזמון

⚠️ **אותו הרגל, מופנה פנימה**

את השאלות שאתם שואלים על פלט של AI — מה הומצא, מה חסר, מה הונח — שאלו על הדרישה עצמה: איזו החלטה עוד לא נאמרה?

המנתח לא מנסח מושלם בניסיון הראשון — הוא יודע איזו שאלה לשאול הבאה. את ההחלטות שנחשוף נקבע בדרישות המפורטות — בשקף הבא.

ניתוח דרישות

SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב · AI = Artificial Intelligence · בינה מלאכותית

הפורמט: דרישה כללית ← דרישות מפורטות

שתי רמות — הכללית אומרת מה מתאפשר; המפורטות קובעות את ההתנהגות המדויקת, SMART ובנות-בדיקה

רמה 1 — דרישה כללית · FR-n · נושאת שחקן: "המערכת תאפשר ל<שחקן> ל<יכולת>."
רמה 2 — דרישות מפורטות · FR-n.1, FR-n.2… : כל אחת משפט SMART מלא — "המערכת ת<פעולה> <תנאי / כמות / תזמון מדידים>."

FR-4 דרישה כללית | שחקן: מנהל

המערכת תאפשר למנהל לנהל את זמינות פריטי התפריט.

הדרישות המפורטות (SMART) — כאן נקבעות ההחלטות שהשאלות חשפו:

FR-4.1 המערכת תאפשר למנהל לסמן פריט תפריט כ"אזל" או להחזירו למצב "זמין".

FR-4.2 המערכת תציג פריט "אזל" בתפריט כ"לא זמין" (מעומעם, ללא אפשרות הוספה לעגלה), ולא תמחק אותו. ← **התשובה ל"מה זה 'להסיר'?"**

FR-4.3 המערכת תשקף שינוי זמינות לסטודנטים תוך 5 שניות מרגע העדכון.

FR-4.4 המערכת תאפשר שינוי זמינות של פריט רק למשתמש בהרשאת "מנהל". ← **התשובה ל"מי מבצע?"**

זה פורמט ההגשה גם בפרויקט: לכל יכולת דרישה כללית אחת, ומתחתיה המפורטות — כל מפורטת ניתנת לבדיקת קבלה.

FR = Functional Requirement · דרישה פונקציונלית · SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

ניתוח דרישות

עדיפות — MoSCoW

לא כל דרישה נכנסת לגרסה הראשונה — העדיפות קובעת מה בפנים ומה מחכה

Must
חובה — ליבת המערכת. בלעדיה אין מוצר.

Should
רצוי — חשוב, אך הגרסה הראשונה שורדת בלעדיו.

Could
אפשרי — נחמד שיהיה, אם יישאר זמן ותקציב.

Won't
לא בגרסה זו — הוחלט במודע לדחות. מתועד, לא נשכח.

עדיפות = תיחום הגרסה הראשונה — לא סדר ביצוע

Must אינו "מה שבונים קודם" אלא מה שחייב להיות בפנים כשמשחררים. סדר העבודה נקבע בתכנון הפיתוח; העדיפות היא החלטת תכולה שמסוכמת עם הלקוח. היא יושבת על הדרישה הכללית, וכל מפורטת יורשת אותה — אלא אם דורסים במפורש: ניהול עובדים (FR-6) היא Must, אבל מחיקת עובד לצמיתות (FR-6.4) יכולה להיות Could — עדיפות שלא נכתבה = עדיפות ההורה.

מהקמפוס — הסולם המלא

- **FR-2** התראת "מוכן לאיסוף" **Must** — בלי זה אין ערך: הסטודנט שוב מחכה ליד הדלפק.
- **FR-1** סינון לפי פרופיל תזונתי **Should** — חשוב, אך אפשר להזמין גם בלעדיו בגרסה הראשונה.
- **FR-7** הזמנה מועדפת בלחיצה אחת **Could** — נחמד שיהיה, אם יישאר זמן ותקציב — וזה בסדר גמור שלא.
- **FR-5** דוח מכירות סוף-יום **Won't** — חשוב למנהל, אך נדחה במודע מהגרסה הראשונה. נשאר ברשימה.

Won't מתועד כמו כל דרישה — ההבדל בין "החלטנו לדחות" ל"שכחנו" הוא כל ההבדל.

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית

מילון נתונים — מגדירים כל שדה פעם אחת

לכל ישות: השדות, הפורמט, החובה והייחודיות — וכל דרישה מפנה לכאן במקום לחזור על הרשימה

הישות: עובד קפיטריה (נשתמש בה בסעיף הדרישות המלא בהמשך)

שדה	ערכים / פורמט	חובה	ייחודי	הערות
שם מלא	טקסט, עברית או אנגלית	✓	—	
שם משתמש	טקסט	✓	✓	נדחה אם כבר קיים במערכת
תפקיד	דלפק / מטבח / מנהל	✓	—	רשימה סגורה — קובע הרשאות
דוא"ל	פורמט תקין	✓	—	
קפיטריה	מרשימת הקפיטריות	✓	—	
סטטוס	פעיל / לא-פעיל	✓	—	ברירת מחדל: פעיל
טלפון	ספרות בלבד	רשות	—	

למה פעם אחת

בלי מילון, כל דרישה חוזרת על רשימת השדות — והרשימות סוטות זו מזו בשקט. שינוי שדה (הוספת "תעודת סטודנט"? פורמט טלפון?) מתעדכן במקום אחד, וכל הדרישות שמפנות אליו מתעדכנות איתו.

שדה = דרישה פונקציונלית

"שם מלא בעברית או באנגלית" היא התנהגות של שדה מסוים — דרישה פונקציונלית ברמת השדה. המילון הוא המקום שבו הדרישות האלו חיות; עוד על ההבחנה מהדרישות הרוחביות — בשקף ה-NFR בהמשך.

📁 מילון מלא לכל ישויות הקפיטריה — בפורמט לפרויקט שלכם — בחומרי הקורס: SAD-course-materials · תיקיית 3-ניתוח-דרישות

מילון הנתונים הוא חלק ממסמך הדרישות: פעולות ה-CRUD בשקפים הבאים מפנות אליו — לא חוזרות עליו.

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה

CRUD — בדיקת שלמות לכל ישות נתונים

ארבע פעולות היסוד על נתונים — ולמה הן רשימת ביקורת שימושית

ארבע פעולות היסוד על נתונים

- C · Create — יצירה (להוסיף רשומה חדשה)
- R · Read — קריאה (להציג / לאחזר נתון קיים)
- U · Update — עדכון (לשנות רשומה קיימת)
- D · Delete — מחיקה (להסיר רשומה)

למה זו בדיקת שלמות

לכל ישות נתונים שְׁאלו: אילו מארבע הפעולות המערכת באמת צריכה? פעולה חסרה מסגירה דרישה שנשכחה — CRUD חושפת חורים שלא ראינו מעצמם.

ובפורמט שלנו: "ניהול X" = דרישה כללית + ארבע מפורטות (FR-n.1…4), אחת לכל פעולה, כל אחת "המערכת ת…".

מטריצת CRUD — מערכת ההזמנות בקמפוס · שני סוגי נתונים

ישות נתונים	Create · יצירה	Read · קריאה	Update · עדכון	Delete · מחיקה
🏠 פריט תפריט (menu item)	המנהל מוסיף מנה חדשה	הסטודנט רואה זמינות ומחיר	המנהל עורך מחיר / זמינות	המנהל מסיר מנה מהתפריט
🏠 הזמנה (order)	הסטודנט יוצר הזמנה	הסטודנט בודק סטטוס	המטבח מסמן 'מוכן'	הסטודנט מבטל הזמנה
🔗 סטודנט (student)	לא אצלנו — ממשק יומי ממערכת הרישום	מציגים שם ומספר סטודנט בהזמנה	רק תוספת מקומית: פרופיל תזונתי — לא נתוני הליבה	לא אצלנו — מי שעזב נעלם בסנכרון הבא

🏠 נתונים בבעלותנו — כל ארבע הפעולות אצלנו · 🔗 ממקור חיצוני (ממשק / יבוא) — אצלנו בעיקר קריאה ותוספות מקומיות; ה-C/U/D קורים במערכת המקור.

🤖 הבינה ממלאת את המטריצה

Claude מקבל את רשימת הישויות וממלא לכל אחת את ארבע הפעולות — ומייצר במהירות דרישות פונקציונליות מועמדות לכל תא.

⚠️ איזה תא היא לא נכון מילאה?

האם כל פעולה שהוצעה באמת נדרשת — ואצלנו? "מחיקת הזמנה" בידי סטודנט היא ביטול, לא מחיקה; ול"סטודנט" ה-AI ישמח להציע Create מלא — למרות שהוא בכלל מגיע ממערכת הרישום.

לכל ישות: קודם מי הבעלים, ואז C·R·U·D — מה שחסר במטריצה הוא דרישה שנשכחה.

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית · CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה · AI = Artificial Intelligence · בינה מלאכותית

סעיף דרישות מלא — ניהול עובדים והרשאות

כך נראה סעיף שלם במסמך: דרישה כללית · הפניה למילון הנתונים · ארבע פעולות CRUD בפורמט המלא

FR-6 Must שחקן: מנהל **המערכת תאפשר למנהל לנהל את עובדי הקמפוס, תפקידיהם והרשאות הגישה שלהם.** — יכולת שאינה קיימת היום

📖 שדות העובד — מוגדרים במילון הנתונים שראינו. כל פעולה מפנה לשם: חובה/רשות, ייחודיות ופורמט כתובים פעם אחת.

הפעולות — כל תת-דרישה נכתבת כ"המערכת ת..." — פעולה + בדיקת תקינות + תוצאה או חריג:

C **Create · יצירה** FR-6.1
- 6.1.1 המערכת תאפשר למנהל להוסיף עובד עם שדות החובה ולשייך לו תפקיד.
- 6.1.2 המערכת תקצה מזהה ייחודי ותדחה שם-משתמש שאינו ייחודי.
- 6.1.3 אם שדה חובה חסר/שגוי — המערכת לא תיצור את הרשומה ו**תציג שגיאה**.

R **Read · קריאה** FR-6.2
- 6.2.1 המערכת תאפשר למנהל לצפות בפרטי עובד.
- 6.2.2 המערכת תאפשר לחפש ולסנן עובדים לפי תפקיד, קפיטריה או סטטוס.
- 6.2.3 המערכת תציג לעובד את הפרופיל שלו בלבד.

U **Update · עדכון** FR-6.3
המערכת תאפשר למנהל לעדכן את פרטי העובד, תפקידו והרשאותיו (למעט המזהה). **במעבר ל"לא-פעיל" המערכת תחסום גישה ותשמור את הרשומה (מחיקה רכה).**

D **Delete · מחיקה** FR-6.4
המערכת תסיר רשומת עובד לצמיתות רק בתנאים מוגדרים (טעות / התיישנות / דרישה חוקית). ← **מחיקה אינה פעולה יומיומית: השגרה היא "לא-פעיל" — עדכון סטטוס.**

מילון + כללית + C/R/U/D = סעיף שלם במסמך · כל פעולה מפורקת לעומק שהיא צריכה — לא תמיד שורה אחת.

FR = Functional Requirement · דרישה פונקציונלית · Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה

ניתוח דרישות

חוקים עסקיים — ומה הקשר לדרישות

חוק עסקי הוא מדיניות של הארגון — הבעלים שלו עסקי ולא טכני · הדרישות הן מי שאוכף אותו

מה זה חוק עסקי (Business Rule)

הצהרת מדיניות או אילוץ של הארגון: מי מוסמך, מה מותר, מתי ובאילו תנאים. המבחן הוא מי הבעלים — החלטה של העסק היא חוק; החלטה של הפיתוח היא דרישה טכנית.

מאיפה החוק מגיע

קיים (AS-IS) — היה כאן לפנינו, לרוב לא כתוב. מגלים ומתעדים.
טרם הוחלט — הארגון הסתדר בעמימות, ותוכנה לא יכולה. מעלים להחלטה.
חדש (TO-BE) — התהליך החדש מביא מדיניות חדשה. צריך אישור.

איך מנהלים אותו

קטלוג נפרד במסמך הדרישות: מזהה (BR-n) + בעלים עסקי שמוסמך לשנותו. חוק משתנה בהחלטת הנהלה — לא בהחלטת מפתח; כשהוא משתנה, מוצאים לפי ההפניות בדיוק אילו דרישות מושפעות.

קטלוג החוקים של הקמפוס — וההפניות מהדרישות

מזהה	החוק העסקי	מקור	בעלים	נאכף ע"י
BR-1	שינוי תפריט, מלאי וזמינות — בסמכות מנהל קפיטריה בלבד.	קיים	הנהלת הקפיטריות	FR-4.4 · FR-6.3
BR-2	הזמנה נמסרת אך ורק למי שהזמין ושילם.	קיים	הנהלת הקפיטריות	FR-3.2
BR-4	ביטול הזמנה מותר עד שהיא עוברת ל"בהכנה" — לא אחריה.	חדש	הנהלת הקפיטריות	FR-3.5
BR-3	הזמנה ששולמה ולא נאספה עד סוף חלון האיסוף — **מדיניות טרם הוחלטה** (זיכוי? חיוב? חסימה?).	טרם הוחלט	אגף שירותי קמפוס	— ממתין להחלטה

חוק אחד יכול להוליד כמה דרישות · BR-4 קיים רק בזכות המערכת — בלי מעקב סטטוס אין רגע לתלות בו את הכלל.

⚠️ שאלת החידוד של האנליסט

מאחורי הרבה דרישות מסתתר חוק עסקי שאיש לא כתב ("רק מנהל משנה זמינות" — כי יש מדיניות סמכויות). BR-3 הוא ההפך: חוק שטרם הוחלט — הדרישה שתאכוף אותו תיכתב רק אחרי שההנהלה תחליט. לא "חור בדרישות" — החלטה עסקית פתוחה, מתועדת.

📁 הקטלוג המלא, אחד עשר החוקים, בשקף הבא · והמסמך עצמו בחומרי הקורס: SAD-course-materials · תיקיית 3-ניתוח-דרישות

דרישה עונה "מה המערכת תעשה" · חוק עסקי עונה "למה" — קטלגו אותו, תנו לו בעלים, והפנו אליו.

FR = Functional Requirement · דרישה פונקציונלית

ניתוח דרישות

הקטלוג המלא: אחד עשר החוקים של הקמפוס

זה מה שנראה קטלוג חוקים בפרויקט שלכם · שלושת המקורות זה לצד זה, ולכל חוק בעלים ומי שאוכף

מזהה	החוק העסקי	מקור	בעלים	נאכף ע"י
BR-1	שינוי תפריט, מלאי וזמינות בסמכות מנהל קפיטריה בלבד.	קיים	הנהלת הקפיטריות	FR-4.4 · FR-6.3
BR-2	הזמנה נמסרת אך ורק למי שהזמין ושילם.	קיים	הנהלת הקפיטריות	FR-3.2
BR-3	הזמנה ששולמה ולא נאספה עד סוף חלון האיסוף: **מדיניות טרם הוחלטה**.	טרם הוחלט	אגף שירותי קמפוס	ממתין להחלטה
BR-4	ביטול הזמנה מותר עד שהיא עוברת ל"בהכנה", לא אחריה.	חדש	הנהלת הקפיטריות	FR-3.5
BR-5	הכנה מתחילה רק לאחר שהתשלום אושר. אין תשלום בדלפק.	חדש	הנהלת הקפיטריות	FR-9.3 · FR-9.4 · FR-10.1
BR-6	חלון האיסוף הוא 60 דקות מרגע סימון "מוכנה".	חדש	אגף שירותי קמפוס	FR-12.1 · FR-2.1
BR-7	הזמנה מתקבלת רק בשעות הפעילות, וקפיטריה "סגורה זמנית" אינה מקבלת הזמנות.	קיים	אגף שירותי קמפוס	FR-8.5 · FR-9.2
BR-8	הזמנה מראש היא לאותו יום בלבד.	חדש	הנהלת הקפיטריות	FR-9.2
BR-9	פריט שאזל אינו נמחק אלא מסומן, ואינו ניתן להזמנה כל עוד הוא מסומן כך.	קיים	מנהל קפיטריה	FR-4.1 · FR-4.2 · FR-11.3
BR-10	רשומות הזמנה ותשלום נשמרות שבע שנים מתום שנת הכספים, ואחריהן נמחקות.	קיים	אגף הכספים	אין FR · נאכף בעיצוב
BR-11	סימון "מוכנה" בסמכות עובד מטבח בלבד, ורק אחרי שכל עמדות ההכנה סיימו.	קיים	מנהל קפיטריה	FR-10.3 · FR-10.4

שתי שורות יוצאות מהתבנית, ובכוונה: BR-10 הוא חוק בלי דרישה פונקציונלית, והוא נאכף בעיצוב, בזנב הארכיון של תרשים המצבים (הרצאה 8) · BR-11 הוא חוק סמכות, ולכן הוא מופיע שם כשומר על המעבר ל"מוכנה", לא כנוסחה.

אחד עשר חוקים, שלושה מקורות, ולכל אחד בעלים שמוסמך לשנות אותו · בהרצאה 8 כל מעבר בתרשים המצבים יצביע על שורה כאן.

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית

דרישות לא-פונקציונליות (NFR) — כמה טוב

איכות ואילוץ שחלים על המערכת כולה — לא פעולה של שחקן מסוים

איך מבחינים? אותה מילה — קנה מידה אחר

✅ פונקציונלי
"שם פרטי ומשפחה יכולים להיות בעברית או באנגלית" — התנהגות של שדה מסוים (ראינו: זה חי במילון הנתונים).

🔶 לא-פונקציונלי
"כל קלט במערכת יכול להיות בעברית או באנגלית" — איכות רוחבית על כל המערכת.

משפחות ה-NFR — ולכל אחת חייב להיות מדד

- ביצועים — זמן תגובה, משך תהליך
- עומס — משתמשים/פעולות במקביל
- זמינות — אחוז זמן שהמערכת למעלה
- אבטחה ופרטיות — אימות, הרשאות, מה לא נשמר
- מדרגיות — גדילה בלי לבנות מחדש
- שמישות — למידה, נגישות, שפות

⛔ משאלה
"המערכת תהיה מהירה ומאובטחת." — בלי מדד, אי-אפשר לבדוק. המילים המסוכנות שוב.

✅ דרישה
"המערכת תשלים תהליך הזמנה מלא תוך ≤ 90 שניות." — מספר, יחידה, תנאי. יש תרחיש בדיקה.

⚠️ למה זה קריטי דווקא עכשיו
ה-NFR הן הדרישות שהכי נשכחות בראיונות — אף בעל-עניין לא "מבקש" עומס שיא. האנליסט שואל עליהן יזומות, וכותב אותן בסעיף נפרד עם מזהה (NFR-n) — הטבלה המלאה של הקמפוס: בשקף הבא.

פונקציונלי = מה המערכת עושה · לא-פונקציונלי = כמה טוב, על כולה — ותמיד עם מספר.

ניתוח דרישות

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית

והלא-פונקציונליות? סעיף נפרד — עם מדד

אילוץ על כל המערכת, לא יכולת של שחקן — ולכן טבלה משלהן, ולכל אחת מספר שאפשר לבדוק

מזהה	קטגוריה	הדרישה — והמדד שלה
NFR-1	**עומס שיא**	500 הזמנות במקביל בין 12:00–14:00, זמן תגובה ≤ 2 שניות.
NFR-2	**ביצועים**	תהליך הזמנה מלא (מבחירה עד אישור תשלום) ≤ 90 שניות.
NFR-3	**אבטחה ופרטיות**	אימות דרך זיהוי האוניברסיטה בלבד; ללא אחסון סיסמאות או פרטי אשראי.
NFR-4	**מדרגיות**	תמיכה ב-5000 סטודנטים.
NFR-5	**זמינות**	99.5% בשעות הפעילות (08:00–20:00).

⛔ **כך זה נראה קודם**

"ממשק פשוט ומהיר — מתאים להפסקה של 15 דקות."

"פשוט" ו"מהיר" בלי מדד — אי-אפשר לבדוק. בדיוק המילים המסוכנות שראינו.

✅ **אותו צורך, עם מדד**

NFR-2: תהליך הזמנה מלא ≤ 90 שניות.

ההפסקה הקצרה הפכה למספר — עכשיו יש תרחיש בדיקה.

📁 סעיף ה-NFR המתועד במלואו — בתוך מסמך הדרישות המלא — בחומרי הקורס: SAD-course-materials · תיקיית 3-ניתוח-דרישות

דרישה לא-פונקציונלית בלי מדד היא משאלה — את טבלת ה-FR המקבילה נבנה בסוף ההרצאה, והשתיים יחד הן הקלט להרצאה 4-5.

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית

ניתוח דרישות

טובה או לא? FR או NFR?

שמונה דרישות מהקמפוס — התווכחו על כל אחת, ואז לחצו להפוך אותו ולראות את התשובה

✅ **FR טובה**

"המערכת ת..." · פועלת + נראית + עושה — יש מה למדוד בדיוק.

⛔ **FR פגומה**

"להיות אינטואיטיבי" — מסובייקטיבי. אין מה, אין בדיקה.

✅ **NFR טובה**

מספרית בדיוק, ומקף: כך מודדים. (זו ה-NFR שלנו.)

⛔ **NFR-פגומה**

"יהיה מהיר" כמה מהיר? אין מזה — אי-אפשר לבדוק שהתקיימה.

⛔ **"אין"**

שתי דרישות בפועל אחד — לא בדוק. או חוכה. לפצל לשתיים.

✅ **FR — בדוקה עכשיו**

לשמש כאן ו-NFR מדי-לשונית, אבל עכשיו ⇐ פונקציונלי. יש בסיס רלוונטי.

⛔ **"אין" בהחלטה**

explainable = מדויק. מדריכה: "מדויק לאחר אחר" — מדריך הוא מחכה ציוד.

✅ **NFR — מדידה-בדקדה**

"כל פעולה" = מקף רחבי, יש מזה (21 מדשים). זו שאנו FR — יכול מדידה, מקף מבריא.

שאלות הבדיקה — לכל כרטיס

נכתבה "המערכת ת..."? יש מדד שאפשר לבדוק? "איך" (מנגנון, טכנולוגיה) התגנב פנימה? וההיקף — שדה/פעולה מסוימת (פונקציונלי) או כל המערכת (לא-פונקציונלי)?

אותן שאלות בדיוק תשאלו על הדרישות של הפרויקט שלכם — ועל כל טיוטה שה-AI יציע.

ניתוח דרישות

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Artificial Intelligence = AI · בינה מלאכותית

User Story — דרישה מנקודת מבט המשתמש

תיאור קצר של צורך, בשפה של מי שישתמש במערכת

הגדרה

סיפור משתמש (User Story) הוא תיאור קצר של דרישה מנקודת המבט של מי שצריך אותה — מי רוצה, מה הוא רוצה, ולמה זה חשוב לו. במקום מפרט פורמלי מנסחים משפט אחד פשוט שכל בעל-עניין מבין.

התבנית

As a [role], I want [capability], so that [value]

"כ[תפקיד] אני רוצה [יכולת] כדי [ערך]"

שלושת החלקים: מי (התפקיד) · מה (היכולת) · למה (הערך שמצדיק את הדרישה).

דוגמה מהקמפוס

"כסטודנט אני רוצה להזמין ארוחה מראש מהטלפון כדי שלא אבזבז את ההפסקה בתור" — ו: "כמנהל קפיטריה אני רוצה לראות את כל ההזמנות בלוח אחד כדי להתמודד עם עומס הצהריים".

User Story — זריז ומשתמש-מרכזי

- מנוסחת בשפת המשתמש, מנקודת מבטו
- קצרה — משפט אחד, לא מסמך
- גמישה ופתוחה למשא ומתן (negotiable)
- מדגישה את הערך — למה הדרישה קיימת
- נפוצה בפיתוח agile / זריז

דרישה קלאסית — פורמלית וחוזית

- מנוסחת בשפה פורמלית של המערכת
- מלאה ומפורטת — שואפת לכסות הכול
- מחייבת וחוזית (contractual) — קשה לשנות
- מתמקדת במה בדיוק המערכת תעשה
- נבדקת מול קריטריונים כמו SMART

דרישה קלאסית עונה מה · User Story מוסיף מי ולמה — והערך מכוון את הדיון.

SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

ניתוח דרישות

הפורמט המורחב — סיפור + תנאי קבלה

משפט הסיפור לבדו אינו בר-בדיקה — תנאי הקבלה (acceptance criteria) הופכים אותו לכזה

> "כסטודנטית, אני רוצה לדעת כשההזמנה שלי מוכנה, כדי שלא אחכה סתם ליד הדלפק בהפסקה הקצרה."

תנאי הקבלה — מתי הסיפור "גמור":

- ☑ כשהמטבח מסמן הזמנה כ"מוכנה" — הסטודנטית מיודעת תוך 10 שניות.
- ☑ אם ההודעה הראשונה לא נמסרה — מתבצע ניסיון גיבוי, והמסירה מתועדת.
- ☑ ההודעה כוללת את מספר ההזמנה ואת נקודת האיסוף.
- ☑ לא נשלחת הודעה על הזמנה שבוטלה.

מוכר לכם?

זה בדיוק המבנה הדו-רמתי: הסיפור = הדרישה הכללית (מי + מה + למה) · תנאי הקבלה = הדרישות המפורטות — כל תנאי בר-בדיקה, כמו FR-n.x. שני פורמטים, אותו עיקרון.

מה עוד נצמד לסיפור

עדיפות (MoSCoW) על הסיפור כולו · הערות ואילוצים (הפניה ל-BR או ל-NFR רלוונטי) · והסיפור עצמו נשאר קצר — הפירוט חי בתנאים, לא במשפט.

📁 סט סיפורי משתמש מלא לקפיטריה, עם תנאי קבלה — בחומרי הקורס: SAD-course-materials · תיקיית 3-ניתוח-דרישות

סיפור בלי תנאי קבלה הוא הבטחה; עם תנאי קבלה — דרישה שאפשר לבדוק. (זה גם FR-2 שלנו, בלבוש אחר.)

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · FR = Functional Requirement · דרישה פונקציונלית

סיפור טוב, סיפור רע — מה דעתכם?

שישה סיפורים מהקמפוס — התווכחו, ואז לחצו על הכרטיס להפוך אותו ולראות את התשובה

✅ סיפור טוב

אדם אמיתי · יכולת ממוקדת · ערך אמיתי שאפשר להגן עליו. ברור מי נהנה ומה הערך.

⛔ לא סיפור

"כלי שמקל על המורים" — מי המורים? מה הכלי עושה? אי אפשר לדעת.

✅ סיפור טוב

מי-מה-איך בפסקה אחת: סטודנט בודד, כלי ספציפי ותוצאה (או "קול ב סטרוט").

⛔ "כאשרכה" אינה משתמש

סיפור בנוי סביב תכונה שכל אחד. זו Z-אף ממושמת — כלום מאיזי מסמוזטיות.

⛔ כלי קהל

"כלי" שווה ערך ניכיות — מעריכי. כמה זה משנה לו? מי נאמר.

⛔ אחרת (Else)

סיפורים ממוקדים. "כלי את נכון" עדיף מכי סיפור אחד — כאשר כלי את ממוקדים.

⛔ ח"אלך" רלק

סכזוכיאיט = נחמסת שיצוב, כא צורך שר מאחאש. ואחרך ("מחיכות") כא נוכש מחזק.

⚠️ שאלות לבדיקה של כל סיפור

מי באמת המשתמש? היכולת ממוקדת או אימפריה? ה"כדי" מוסיף ערך אמיתי — או חוזר על היכולת? והאם "איך" (מנגנון, טכנולוגיה) התגנב למשפט?

סיפור טוב: אדם אמיתי · יכולת ממוקדת · ערך שאפשר להגן עליו · ובלי "איך".

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית

איך מוודאים שהדרישות 'שלמות'?

בדיקת שלמות ועקביות — לפני שעוברים לאפיון

צ'ק-ליסט השלמות

- כיסוי פונקציונלי ולא-פונקציונלי — יש דרישות גם למה המערכת עושה (FR) וגם לכמה טוב (NFR), ולא רק לאחד מהם.
- כל בעל עניין מטופל — לכל שחקן (סטודנט, מנהל קפיטריה, מטבח) יש דרישות שמכסות את מה שהוא צריך מהמערכת.
- לכל צעד בתהליך יש דרישה — עוברים על תהליך ה־as-is שמיפינו (הרצאה 2), וכל פעולה בו מקבלת דרישה במערכת החדשה.
- אין סתירות — שתי דרישות לא דורשות דברים מנוגדים (עקביות).
- כל דרישה ניתנת לבדיקה ולמעקב — אפשר לבדוק אם התקיימה (testable) ולעקוב ממנה לצורך שממנו נולדה (traceable) — המזהה (FR-n) מאפשר את המעקב.
- כיסוי CRUD לכל ישות — לכל ישות מרכזית (הזמנה, פריט תפריט, סטודנט) יש דרישות ליצירה, קריאה, עדכון ומחיקה.

⚠️ הפערים שהכי שוכחים

בדרך כלל מכוסה דווקא הFR: הדרישות הלא-פונקציונליות (NFR) נשכחות — עומס, אבטחה, פרטיות, מהירות. גם מקרי קצה נופלים בין הכיסאות: מה קורה כשהתשלום נכשל, כשפריט אזל, כשהמטבח לא סימן 'מוכן'? "שלם" אומר גם את המסלולים שאינם רגילים.

🤖 הבינה כבודקת כיסוי

Claude יכול להצליב את רשימת הדרישות מול השחקנים, צעדי התהליך וטבלת ה־CRUD ולסמן מה חסר — אבל ההחלטה מה באמת בגבולות המערכת נשארת שלכם.

דרישות 'שלמות' = כל שחקן, כל צעד וכל ישות מכוסים — בלי סתירות, ועם ה־NFR שלא נשכחו.

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה

ניתוח דרישות

הדרישות של מערכת הקמפוס — רשימה למעקב

מהראיונות אל הטבלה: לכל דרישה כללית מזהה, עדיפות MoSCoW ושחקן-מקור

מזהה · עדיפות	הדרישה הכללית	שחקן
FR-1 Should	המערכת תאפשר לסטודנט לסנן את התפריט לפי פרופיל תזונתי (צמחוני / טבעוני / ללא-גלוטן).	סטודנט
FR-2 Must	המערכת תודיע לסטודנט כשההזמנה מוכנה לאיסוף.	סטודנט
FR-3 Must	המערכת תאפשר לעובד הדלפק לאמת ולמסור הזמנה בקוד איסוף.	עובד דלפק
FR-4 Must	המערכת תאפשר למנהל לנהל את זמינות פריטי התפריט.	מנהל
FR-5 Won't	המערכת תפיק למנהל דוח מכירות סוף-יום.	מנהל
FR-6 Must	המערכת תאפשר למנהל לנהל את עובדי הקמפוס, תפקידיהם והרשאות הגישה שלהם.	מנהל
FR-7 Could	המערכת תאפשר לסטודנט לשמור הזמנה מועדפת ולהזמין אותה שוב בלחיצה אחת.	סטודנט

נדגו מראיונות הסטודנט, המנהל, המטבח והדלפק · מתחת לכל דרישה כללית: המפורטות שלה (FR-n.x), כפי שפירטנו את FR-4.

⚠ הטבלה היא משטח האימות
מה נקבר ברעש הראיונות (אזהרות אלרגיה? נגישות?), מה הומצא ואינו בגבולות המערכת (משלוח למעונות), והיכן הראיונות סותרים זה את זה (תשלום, הזמנות מראש)? — על הרשימה הזו שואלים את שאלות החידוד.

📁 הרשימה נמשכת — דרישות הליבה FR-8…FR-12 (עיון, הזמנה ותשלום, מטבח ועוד) במסמך הדרישות המלא, בחומרי הקורס: SAD-course-materials · תיקיית 3-ניתוח-דרישות

המזהה מלווה את הדרישה לאורך הדרך: מעקב → בדיקות קבלה → התרחיש שימש אותה (הרצאה 4-5).

ניתוח דרישות

FR = Functional Requirement · דרישה פונקציונלית

AI בשלב הדרישות

מהערות ראיון מבולגנות לרשימת דרישות מסווגת — ואז מאתגרים

מה הבינה עושה כאן

אספנו הערות ראיון גולמיות מסטודנטים, מהמטבח ומהמנהל — טקסט חופשי ומבולגן. Claude הופך את הערימה הזו לרשימת דרישות מסודרת תוך שניות: ממיין, מנסח, ומסמן מה חסר. הקלט שלכם הוא הראיונות; הפלט הוא טיוטה שצריך לבדוק.

ממיין FR מול NFR

לוקח את ההערות החופשיות וממפה כל דרישה לפונקציונלית (מה המערכת עושה) או לא-פונקציונלית (כמה טוב), בשתי עמודות.

מנסח בפורמט המלא

טיוטת דרישה כללית + מפורטות SMART עם מזהים (FR-n.x) והצעת עדיפות — במקום "שתהיה מהירה", "תוך 2 שניות בשעת העומס".

מנסח user stories

הופך כל צורך ל"כסטודנט, אני רוצה... כדי...", ומסמן דרישות שנראה שחסרות או שלא נשאלו עליהן בראיון.

מה Claude מטייט

רשימת דרישות מסווגת (NFR/FR), ניסוחי SMART, user stories מתוך הראיונות — וסימון פערים. כל זה תוך שניות, מטקסט גולמי.

⚠ מה צריך לבדוק

היא עלולה לסווג NFR/FR לא נכון, לנסח דרישה מעורפלת או מפורטת-יתר-על-המידה, להמציא דרישה שאיש לא ביקש, או לפספס צורך לא-פונקציונלי שקבור בתוך הטקסט (אבטחה, עומס, פרטיות). הסיווג והניסוח שלכם הם משטח האימות.

הבינה מטייטת — אתם מאשרים. בהרצאה הבאה ניקח את טבלת המעקב הזו והדרישות הכלליות יהפכו לתרחישי use case.

ניתוח דרישות

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · FR = Functional Requirement · דרישה פונקציונלית · SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב · AI = Artificial Intelligence · בינה מלאכותית

תרגיל בית (רשות): מראיון לדרישות

מראיינים בעל-עניין AI אמיתי-לגמרי-כמעט, ומכינים את מה שאנליסט מגיש

🎭 בכלי הקורס (sad-mcp): בקשו "אני רוצה לתרגל ראיון עם שמעון" — מנהל הקפיטריה. הוא עונה בעברית, מתגונן, ומערבב עיקר וטפל. עומק מקבלים רק על שאלות טובות.

א • מכינים

- דף שאלות מראש: BSP (בעיות + החלטות) · CSF · E/M — כמו בשקף ההכנה לקמפוס.
- ההכנה היא חצי מהעבודה.

ב • מראיינים

- משפך · "למה" · לבקש מספרים · ללחוץ על סתירות (יש לו אחת...).
- בסיום: [סיכום] ← משוב מאמן אישי, לא ציון. שמרו אותו והשוו אליו בראיון הבא.

ג • כותבים

- טבלת מעקב: 4–6 דרישות כלליות (מזהה · MoSCoW · שחקן), אחת לפחות מפורקת ל-FR-n.x.
- 2+ NFR עם מדד · חוק עסקי עם בעלים · שאלה פתוחה אחת "ממתין להחלטה".

⭐ בונוס — הראיון השני

ראיינו גם את דנה, עובדת הדלפק (persona: dana) וכתבו 3 פערים בין מה ששמעון אמר למה שהיא אמרה, ומה כל פער מלמד על הדרישות. רמז מהשיעור: הדרישה הכי חשובה לפעמים דבוקה בפתק על הקופה.

רשות, לא להגשה. תרגילי הבית להגשה ניתנים במודל. ותזכרו את Wetherbe: שמעון לא יודע מה הוא צריך — זו בדיוק העבודה שלכם.

ניתוח דרישות

CSF = Critical Success Factors · גורמי הצלחה קריטיים · NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · FR = Functional Requirement · דרישה פונקציונלית · AI = Artificial Intelligence · בינה מלאכותית

סיכום — והבא בתור

מה למדנו על דרישות, ולאן זה ממשיך

מהן דרישות

- פונקציונליות (FR) — מה המערכת עושה
- לא-פונקציונליות (NFR) — כמה טוב היא עושה זאת

למה זה קריטי

- ככל שמתקנים מאוחר יותר — העלות גבוהה יותר
- דרישה שגויה או חסרה = מערכת שעונה על הבעיה הלא-נכונה

זיהוי צרכים (Wetherbe)

- מנהלים לא יודעים מה הם צריכים — 4 טעויות ← 4 פתרונות
- חוצה-פונקציות · JAD · ראיון מובנה (שאלות עקיפות) · אב-טיפוס

ניסוח טוב

- "המערכת ת..." · SMART — מדידה ובת-בדיקה, בלי מילים מעורפלות
- וגם אז לא "גמורה" — לשאול איזו החלטה עוד לא נאמרה

הפורמט והשלמות

- כללית ← מפורטות (FR-n.x) · עדיפות MoSCoW · User Story · CRUD
- בדיקת שלמות — כל שחקן, כל ישות, וה-NFR עם מדד

🤖 בינה בכל שלב

- Claude מנסח, מסווג ומכין דרישות במהירות
- ⚠️ אנחנו מי שמוודאים — מה היא פספסה, המציאה או הניחה

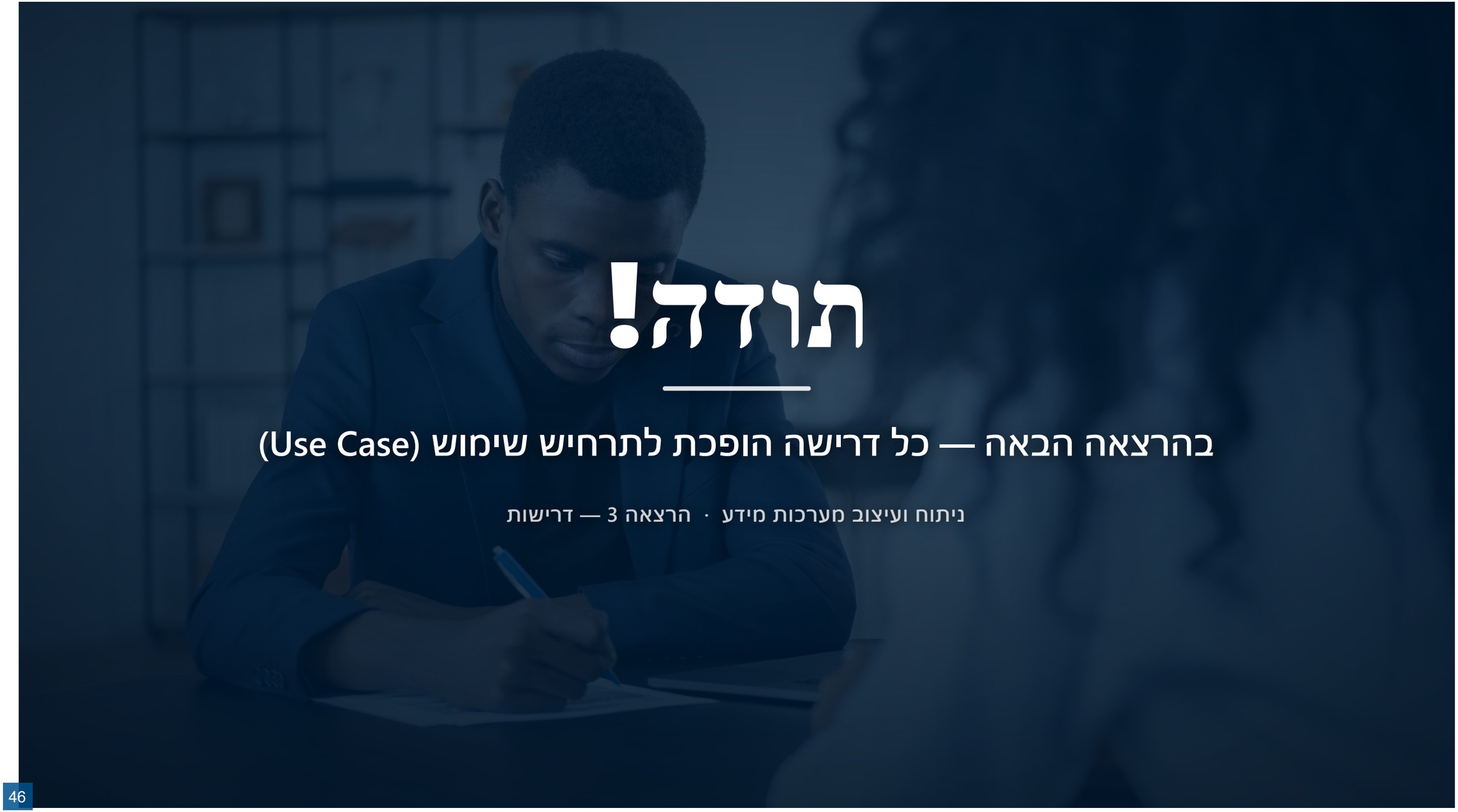
הבא בתור — מדרישות למודל: תרשים Use Case (הרצאות 4–5)

יש לנו טבלת מעקב: דרישות כלליות עם מזהים, שחקן ועדיפות, ומתחתן המפורטות. בשלב הבא כל דרישה כללית תהפוך לתרחיש שימוש (שחקן + מטרה) בתרשים Use Case — מי השחקנים, מה גבול המערכת, ומה כל אחד עושה בה. עדיין אותו שלב — ניתוח דרישות — רק מייצגים אותו ויזואלית.

דרישה טובה היא דרישה שאפשר לבדוק — בהרצאה הבאה נצייר את הדרישות הפונקציונליות כתרשים Use Case.

NFR = Non-Functional Requirement · דרישה לא-פונקציונלית · FR = Functional Requirement · דרישה פונקציונלית · SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב · CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה · JAD = Joint Application Development · פיתוח יישומים משותף

ניתוח דרישות


תודה!
בהרצאה הבאה — כל דרישה הופכת לתרחיש שימוש (Use Case)
ניתוח ועיצוב מערכות מידע · הרצאה 3 — דרישות